

Variables Cuantitativas

July 13, 2026

3_1_4_var_cuantitativas

July 13, 2026

PAO26-26 - Variables Cuantitativas

Kevin Xavier Suasnavas Villarreal

Docente: Elvis Pachacama

Link a mi GitHub:

<https://github.com/kevinsuas433/Machine2026>

1 0. Variables Cuantitativas

Las **variables cuantitativas** representan características que pueden medirse mediante valores numéricos. Estas variables permiten realizar operaciones matemáticas y obtener diferentes medidas estadísticas para describir el comportamiento de los datos.

En esta práctica se analizarán diversas medidas de tendencia central utilizando Python, NumPy y SciPy.

1.1 Creación del conjunto de datos

En este ejemplo se crea un arreglo numérico que contiene valores positivos, negativos y decimales. Posteriormente se utilizará este conjunto de datos para calcular las principales medidas de tendencia central.

```
[3]: # =====  
# Importar la biblioteca NumPy.  
# =====  
  
import numpy as np  
  
# =====  
# Crear un arreglo con valores numéricos.  
# =====  
  
X = np.array([
```

```

    0.5, 23, 0.3, 4.5,
    0.3, 0.5, -28, -50,
    60, -100, -10, -11,
    13, 19, 1, 9
])

# Mostrar el contenido del arreglo.
print("Datos del arreglo:")
display(X)

```

Datos del arreglo:

```

array([  0.5,  23. ,   0.3,   4.5,   0.3,   0.5, -28. , -50. ,
        60. , -100. , -10. , -11. ,  13. ,  19. ,   1. ,   9. ])

```

1.2 Medidas de tendencia central

Las medidas de tendencia central permiten resumir un conjunto de datos mediante un valor representativo.

Las principales medidas son:

- **Media:** promedio de todos los valores.
- **Mediana:** valor central del conjunto de datos ordenado.
- **Moda:** valor que aparece con mayor frecuencia.

```

[4]: # =====
# Importar el módulo de estadísticas de SciPy.
# =====

from scipy import stats

# =====
# Calcular las medidas de tendencia central.
# =====

# Calcular la media aritmética.
media = np.mean(X)

# Calcular la mediana.
mediana = np.median(X)

# Calcular la moda.
moda, _ = stats.mode(X)

# Mostrar los resultados obtenidos.
print(f"Media: {media}")

print(f"Mediana: {mediana}")

```

```
print(f"Moda: {moda}")
```

Media: -4.24375

Mediana: 0.5

Moda: 0.3

Interpretación de los resultados

- La **media** corresponde al promedio de todos los valores del conjunto de datos.
- La **mediana** representa el valor central cuando los datos se ordenan de menor a mayor.
- La **moda** identifica el valor que más veces se repite dentro del conjunto de datos.

Estas medidas permiten comprender el comportamiento general de una variable cuantitativa y constituyen la base del análisis estadístico descriptivo.

1.3 Medidas de posición

Las **medidas de posición** permiten dividir un conjunto de datos ordenado en diferentes partes iguales, facilitando el análisis de la distribución de los valores.

En esta práctica se calcularán:

- **Primer cuartil (Q1):** corresponde al 25 % de los datos.
- **Tercer cuartil (Q3):** corresponde al 75 % de los datos.
- **Rango Intercuartílico (RIC):** diferencia entre el tercer y el primer cuartil.
- **Límites inferior y superior:** utilizados para detectar posibles valores atípicos (*outliers*).

```
[5]: # =====  
# Importar las bibliotecas necesarias.  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
# =====  
# Calcular los cuartiles.  
# =====  
  
# Calcular el primer cuartil (25 %).  
Q1 = np.percentile(X, 25)  
  
# Calcular el tercer cuartil (75 %).  
Q3 = np.percentile(X, 75)  
  
# =====  
# Calcular el rango intercuartílico (RIC).  
# =====
```

```

RIC = Q3 - Q1

# =====
# Calcular los límites para detectar valores atípicos.
# =====

lim_inf = Q1 - 1.5 * RIC

lim_sup = Q3 + 1.5 * RIC

# =====
# Mostrar los resultados obtenidos.
# =====

print(f"Primer cuartil (Q1): {Q1}")

print(f"Tercer cuartil (Q3): {Q3}")

print(f"Rango intercuartílico (RIC): {RIC}")

print(f"Límite inferior: {lim_inf}")

print(f"Límite superior: {lim_sup}")

```

```

Primer cuartil (Q1): -10.25
Tercer cuartil (Q3): 10.0
Rango intercuartílico (RIC): 20.25
Límite inferior: -40.625
Límite superior: 40.375

```

1.3.1 Diagrama de caja y bigotes (Boxplot)

El **diagrama de caja y bigotes** permite visualizar la distribución de un conjunto de datos.

Este gráfico muestra:

- La mediana.
- Los cuartiles.
- El rango intercuartílico.
- Los valores mínimos y máximos.
- Los posibles valores atípicos (*outliers*), representados como puntos fuera de los límites establecidos.

```

[6]: # =====
# Crear el diagrama de caja y bigotes.
# =====

# Crear una figura con un tamaño adecuado.
plt.figure(figsize=(8, 5))

```

```

# Dibujar el diagrama de caja.
plt.boxplot(
    X,
    patch_artist=True,
    boxprops=dict(facecolor="lightblue", color="black"),
    medianprops=dict(color="red", linewidth=2),
    whiskerprops=dict(color="black"),
    capprops=dict(color="black"),
    flierprops=dict(
        marker="o",
        markerfacecolor="red",
        markersize=7,
        linestyle="none"
    )
)

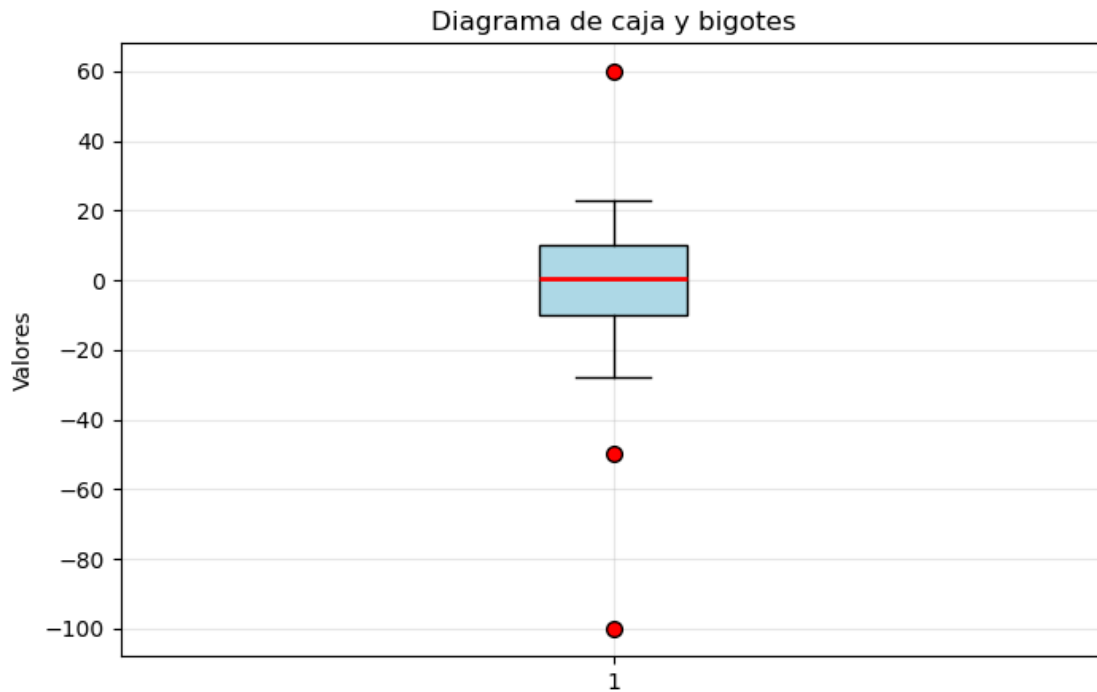
# Agregar una cuadrícula.
plt.grid(alpha=0.3)

# Agregar un título.
plt.title("Diagrama de caja y bigotes")

# Etiqueta del eje Y.
plt.ylabel("Valores")

# Mostrar el gráfico.
plt.show()

```



Interpretación del gráfico

El diagrama de caja y bigotes resume gráficamente la distribución de los datos mediante los cuartiles y la mediana.

Los puntos que aparecen fuera de los límites de los bigotes corresponden a **valores atípicos (outliers)**, es decir, observaciones que se alejan significativamente del resto de los datos.

En este conjunto de datos, los límites calculados mediante el **Rango Intercuartílico (RIC)** permiten identificar estos valores extremos de forma visual.

1.4 Medidas de dispersión

Las **medidas de dispersión** indican qué tan separados se encuentran los datos respecto a su valor central.

Las más utilizadas son:

- **Desviación estándar:** mide cuánto se alejan, en promedio, los datos de la media.
- **Varianza:** representa el cuadrado de la desviación estándar y cuantifica la variabilidad del conjunto de datos.

```
[7]: # =====
# Calcular las medidas de dispersión.
# =====
```

```

# Calcular la desviación estándar.
desvest = np.std(X)

# Calcular la varianza.
varianza = np.var(X)

# Mostrar los resultados.
print(f"Desviación estándar: {desvest:.4f}")
print(f"Varianza: {varianza:.4f}")

```

Desviación estándar: 33.5212
Varianza: 1123.6737

1.5 Medidas de distribución

Las medidas de distribución permiten analizar la forma que presenta un conjunto de datos.

En esta práctica se calcularán:

- **Asimetría (Skewness):** indica si la distribución presenta sesgo hacia la izquierda o hacia la derecha.
- **Curtosis (Kurtosis):** describe el grado de concentración de los datos alrededor de la media.

Según el valor de la curtosis, la distribución puede clasificarse como:

- **Leptocúrtica:** curtosis mayor que cero.
- **Mesocúrtica:** curtosis igual a cero.
- **Platicúrtica:** curtosis menor que cero.

```

[8]: # =====
# Calcular las medidas de distribución.
# =====

# Calcular la asimetría.
asimetria = stats.skew(X)

# Calcular la curtosis.
curtosis = stats.kurtosis(X, fisher=True)

# Mostrar los resultados.
print(f"Asimetría: {asimetria:.4f}")
print(f"Curtosis: {curtosis:.4f}")

# Determinar el tipo de distribución.
if curtosis > 0:
    print("Distribución Leptocúrtica")

elif curtosis < 0:
    print("Distribución Platicúrtica")

```

```
else:
    print("Distribución Mesocúrtica")
```

Asimetría: -1.1302

Curtosis: 2.2858

Distribución Leptocúrtica

Interpretación

- Una **asimetría negativa** indica que la cola de la distribución se extiende hacia la izquierda.
- Una **asimetría positiva** indica que la cola de la distribución se extiende hacia la derecha.
- Una distribución **leptocúrtica** presenta una mayor concentración de datos alrededor de la media y colas más pronunciadas que una distribución normal.

1.6 Análisis del conjunto de datos Iris

En esta sección se utilizará el conjunto de datos **Iris**, incluido en la biblioteca **Scikit-Learn**.

Este dataset contiene **150 observaciones** correspondientes a tres especies de flores Iris y cuatro variables numéricas:

- Longitud del sépalo.
- Ancho del sépalo.
- Longitud del pétalo.
- Ancho del pétalo.

Se analizarán la **asimetría** de la longitud del pétalo y la **curtosis** del ancho del sépalo.

```
[9]: # =====
# Cargar el conjunto de datos Iris.
# =====

from sklearn import datasets

# Cargar el dataset.
iris = datasets.load_iris()

# Obtener las variables predictoras y las clases.
X = iris.data
y = iris.target

# Mostrar información general del dataset.
print("Variables:")
print(iris.feature_names)

print()

print("Clases:")
```



```

print(iris.target_names)

print()

print(f"Número de observaciones: {X.shape[0]}")
print(f"Número de variables: {X.shape[1]}")

```

Variables:

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

Clases:

```
['setosa' 'versicolor' 'virginica']
```

Número de observaciones: 150

Número de variables: 4

```

[10]: # =====
# Calcular la asimetría de la longitud del pétalo.
# =====

# Columna correspondiente a la longitud del pétalo.
petal_length = X[:, 2]

# Calcular la asimetría.
asimetria_petal = stats.skew(petal_length)

print(f"Asimetría (longitud del pétalo): {asimetria_petal:.4f}")

# Interpretar el resultado.
if asimetria_petal > 0:
    print("La variable presenta una asimetría positiva (sesgo hacia la derecha).
    ↪")

elif asimetria_petal < 0:
    print("La variable presenta una asimetría negativa (sesgo hacia la
    ↪izquierda).")

else:
    print("La distribución es simétrica.")

```

Asimetría (longitud del pétalo): -0.2721

La variable presenta una asimetría negativa (sesgo hacia la izquierda).

```

[11]: # =====
# Calcular la curtosis del ancho del sépalo.
# =====

```

```

# Columna correspondiente al ancho del sépalo.
sepal_width = X[:, 1]

# Calcular la curtosis.
curtosis_sepal = stats.kurtosis(
    sepal_width,
    fisher=True
)

print(f"Curtosis (ancho del sépalo): {curtosis_sepal:.4f}")

# Determinar el tipo de distribución.
if curtosis_sepal > 0:
    print("La distribución es leptocúrtica.")

elif curtosis_sepal < 0:
    print("La distribución es platicúrtica.")

else:
    print("La distribución es mesocúrtica.")

```

Curtosis (ancho del sépalo): 0.1810
 La distribución es leptocúrtica.

```

[12]: # Cargar el set de datos de Iris
from sklearn import datasets
from pprint import pprint
iris = datasets.load_iris()
pprint(iris)
X = iris.data
y = iris.target
# ¿Qué tipo de asimetría se observa en la variable "longitud del pétalo"?
#???
# ¿A qué tipo de distribución se ajusta la variable "ancho del sépalo", según
su curtosis?
#???

```

```

{'DESCR': '.. _iris_dataset:\n'
'\n'
'Iris plants dataset\n'
'-----\n'
'\n'
'**Data Set Characteristics:**\n'
'\n'
':Number of Instances: 150 (50 in each of three classes)\n'
':Number of Attributes: 4 numeric, predictive attributes and the '
'class\n'
':Attribute Information:\n'

```

```

'    - sepal length in cm\n'
'    - sepal width in cm\n'
'    - petal length in cm\n'
'    - petal width in cm\n'
'    - class:\n'
'        - Iris-Setosa\n'
'        - Iris-Versicolour\n'
'        - Iris-Virginica\n'
'\n'
':Summary Statistics:\n'
'\n'
'===== \n'
'          Min  Max   Mean   SD   Class Correlation\n'
'===== \n'
'sepal length:  4.3  7.9   5.84   0.83    0.7826\n'
'sepal width:   2.0  4.4   3.05   0.43   -0.4194\n'
'petal length:  1.0  6.9   3.76   1.76    0.9490 (high!)\n'
'petal width:   0.1  2.5   1.20   0.76    0.9565 (high!)\n'
'===== \n'
'\n'
':Missing Attribute Values: None\n'
':Class Distribution: 33.3% for each of 3 classes.\n'
':Creator: R.A. Fisher\n'
':Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n'
':Date: July, 1988\n'
'\n'
'The famous Iris database, first used by Sir R.A. Fisher. The '
'dataset is taken\n'
'from Fisher's paper. Note that it's the same as in R, but not as in "
'the UCI\n'
'Machine Learning Repository, which has two wrong data points.\n'
'\n'
'This is perhaps the best known database to be found in the\n'
'pattern recognition literature. Fisher's paper is a classic in the "
'field and\n'
'is referenced frequently to this day. (See Duda & Hart, for '
'example.) The\n'
'data set contains 3 classes of 50 instances each, where each class '
'refers to a\n'
'type of iris plant. One class is linearly separable from the other '
'2; the\n'
'latter are NOT linearly separable from each other.\n'
'\n'
'.. dropdown:: References\n'
'\n'
'    - Fisher, R.A. "The use of multiple measurements in taxonomic '
'problems"\n'
'    Annual Eugenics, 7, Part II, 179-188 (1936); also in '

```

```

    "Contributions to\n'
    '    Mathematical Statistics" (John Wiley, NY, 1950).\n'
    '    - Duda, R.O., & Hart, P.E. (1973) Pattern Classification and '
    'Scene Analysis.\n'
    '    (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page '
    '218.\n'
    '    - Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New '
    'System\n'
    '    Structure and Classification Rule for Recognition in Partially '
    'Exposed\n'
    '    Environments". IEEE Transactions on Pattern Analysis and '
    'Machine\n'
    '    Intelligence, Vol. PAMI-2, No. 1, 67-71.\n'
    '    - Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE '
    'Transactions\n'
    '    on Information Theory, May 1972, 431-433.\n'
    '    - See also: 1988 MLC Proceedings, 54-64. Cheeseman et al"s '
    'AUTOCLASS II\n'
    '    conceptual clustering system finds 3 classes in the data.\n'
    '    - Many, many more ...'\n'
'data': array([[5.1, 3.5, 1.4, 0.2],
    [4.9, 3. , 1.4, 0.2],
    [4.7, 3.2, 1.3, 0.2],
    [4.6, 3.1, 1.5, 0.2],
    [5. , 3.6, 1.4, 0.2],
    [5.4, 3.9, 1.7, 0.4],
    [4.6, 3.4, 1.4, 0.3],
    [5. , 3.4, 1.5, 0.2],
    [4.4, 2.9, 1.4, 0.2],
    [4.9, 3.1, 1.5, 0.1],
    [5.4, 3.7, 1.5, 0.2],
    [4.8, 3.4, 1.6, 0.2],
    [4.8, 3. , 1.4, 0.1],
    [4.3, 3. , 1.1, 0.1],
    [5.8, 4. , 1.2, 0.2],
    [5.7, 4.4, 1.5, 0.4],
    [5.4, 3.9, 1.3, 0.4],
    [5.1, 3.5, 1.4, 0.3],
    [5.7, 3.8, 1.7, 0.3],
    [5.1, 3.8, 1.5, 0.3],
    [5.4, 3.4, 1.7, 0.2],
    [5.1, 3.7, 1.5, 0.4],
    [4.6, 3.6, 1. , 0.2],
    [5.1, 3.3, 1.7, 0.5],
    [4.8, 3.4, 1.9, 0.2],
    [5. , 3. , 1.6, 0.2],
    [5. , 3.4, 1.6, 0.4],
    [5.2, 3.5, 1.5, 0.2],

```

[5.2, 3.4, 1.4, 0.2],
 [4.7, 3.2, 1.6, 0.2],
 [4.8, 3.1, 1.6, 0.2],
 [5.4, 3.4, 1.5, 0.4],
 [5.2, 4.1, 1.5, 0.1],
 [5.5, 4.2, 1.4, 0.2],
 [4.9, 3.1, 1.5, 0.2],
 [5. , 3.2, 1.2, 0.2],
 [5.5, 3.5, 1.3, 0.2],
 [4.9, 3.6, 1.4, 0.1],
 [4.4, 3. , 1.3, 0.2],
 [5.1, 3.4, 1.5, 0.2],
 [5. , 3.5, 1.3, 0.3],
 [4.5, 2.3, 1.3, 0.3],
 [4.4, 3.2, 1.3, 0.2],
 [5. , 3.5, 1.6, 0.6],
 [5.1, 3.8, 1.9, 0.4],
 [4.8, 3. , 1.4, 0.3],
 [5.1, 3.8, 1.6, 0.2],
 [4.6, 3.2, 1.4, 0.2],
 [5.3, 3.7, 1.5, 0.2],
 [5. , 3.3, 1.4, 0.2],
 [7. , 3.2, 4.7, 1.4],
 [6.4, 3.2, 4.5, 1.5],
 [6.9, 3.1, 4.9, 1.5],
 [5.5, 2.3, 4. , 1.3],
 [6.5, 2.8, 4.6, 1.5],
 [5.7, 2.8, 4.5, 1.3],
 [6.3, 3.3, 4.7, 1.6],
 [4.9, 2.4, 3.3, 1.],
 [6.6, 2.9, 4.6, 1.3],
 [5.2, 2.7, 3.9, 1.4],
 [5. , 2. , 3.5, 1.],
 [5.9, 3. , 4.2, 1.5],
 [6. , 2.2, 4. , 1.],
 [6.1, 2.9, 4.7, 1.4],
 [5.6, 2.9, 3.6, 1.3],
 [6.7, 3.1, 4.4, 1.4],
 [5.6, 3. , 4.5, 1.5],
 [5.8, 2.7, 4.1, 1.],
 [6.2, 2.2, 4.5, 1.5],
 [5.6, 2.5, 3.9, 1.1],
 [5.9, 3.2, 4.8, 1.8],
 [6.1, 2.8, 4. , 1.3],
 [6.3, 2.5, 4.9, 1.5],
 [6.1, 2.8, 4.7, 1.2],
 [6.4, 2.9, 4.3, 1.3],
 [6.6, 3. , 4.4, 1.4],

[6.8, 2.8, 4.8, 1.4],
 [6.7, 3. , 5. , 1.7],
 [6. , 2.9, 4.5, 1.5],
 [5.7, 2.6, 3.5, 1.],
 [5.5, 2.4, 3.8, 1.1],
 [5.5, 2.4, 3.7, 1.],
 [5.8, 2.7, 3.9, 1.2],
 [6. , 2.7, 5.1, 1.6],
 [5.4, 3. , 4.5, 1.5],
 [6. , 3.4, 4.5, 1.6],
 [6.7, 3.1, 4.7, 1.5],
 [6.3, 2.3, 4.4, 1.3],
 [5.6, 3. , 4.1, 1.3],
 [5.5, 2.5, 4. , 1.3],
 [5.5, 2.6, 4.4, 1.2],
 [6.1, 3. , 4.6, 1.4],
 [5.8, 2.6, 4. , 1.2],
 [5. , 2.3, 3.3, 1.],
 [5.6, 2.7, 4.2, 1.3],
 [5.7, 3. , 4.2, 1.2],
 [5.7, 2.9, 4.2, 1.3],
 [6.2, 2.9, 4.3, 1.3],
 [5.1, 2.5, 3. , 1.1],
 [5.7, 2.8, 4.1, 1.3],
 [6.3, 3.3, 6. , 2.5],
 [5.8, 2.7, 5.1, 1.9],
 [7.1, 3. , 5.9, 2.1],
 [6.3, 2.9, 5.6, 1.8],
 [6.5, 3. , 5.8, 2.2],
 [7.6, 3. , 6.6, 2.1],
 [4.9, 2.5, 4.5, 1.7],
 [7.3, 2.9, 6.3, 1.8],
 [6.7, 2.5, 5.8, 1.8],
 [7.2, 3.6, 6.1, 2.5],
 [6.5, 3.2, 5.1, 2.],
 [6.4, 2.7, 5.3, 1.9],
 [6.8, 3. , 5.5, 2.1],
 [5.7, 2.5, 5. , 2.],
 [5.8, 2.8, 5.1, 2.4],
 [6.4, 3.2, 5.3, 2.3],
 [6.5, 3. , 5.5, 1.8],
 [7.7, 3.8, 6.7, 2.2],
 [7.7, 2.6, 6.9, 2.3],
 [6. , 2.2, 5. , 1.5],
 [6.9, 3.2, 5.7, 2.3],
 [5.6, 2.8, 4.9, 2.],
 [7.7, 2.8, 6.7, 2.],
 [6.3, 2.7, 4.9, 1.8],

```

[6.7, 3.3, 5.7, 2.1],
[7.2, 3.2, 6. , 1.8],
[6.2, 2.8, 4.8, 1.8],
[6.1, 3. , 4.9, 1.8],
[6.4, 2.8, 5.6, 2.1],
[7.2, 3. , 5.8, 1.6],
[7.4, 2.8, 6.1, 1.9],
[7.9, 3.8, 6.4, 2. ],
[6.4, 2.8, 5.6, 2.2],
[6.3, 2.8, 5.1, 1.5],
[6.1, 2.6, 5.6, 1.4],
[7.7, 3. , 6.1, 2.3],
[6.3, 3.4, 5.6, 2.4],
[6.4, 3.1, 5.5, 1.8],
[6. , 3. , 4.8, 1.8],
[6.9, 3.1, 5.4, 2.1],
[6.7, 3.1, 5.6, 2.4],
[6.9, 3.1, 5.1, 2.3],
[5.8, 2.7, 5.1, 1.9],
[6.8, 3.2, 5.9, 2.3],
[6.7, 3.3, 5.7, 2.5],
[6.7, 3. , 5.2, 2.3],
[6.3, 2.5, 5. , 1.9],
[6.5, 3. , 5.2, 2. ],
[6.2, 3.4, 5.4, 2.3],
[5.9, 3. , 5.1, 1.8]]),
'data_module': 'sklearn.datasets.data',
'feature_names': ['sepal length (cm)',
                  'sepal width (cm)',
                  'petal length (cm)',
                  'petal width (cm)'],
'filename': 'iris.csv',
'frame': None,
'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10')}
```

Type: float64
String form: 2.285789536446141
File: c:\users\kevin\anaconda3\lib\site-packages\numpy__init__.py
Docstring:
Double-precision floating-point number type, compatible with Python

```
:class:`float` and C ``double``.
```

```
:Character code: ``'d'``
```

```
:Canonical name: `numpy.double`
```

```
:Alias on this platform (win32 AMD64): `numpy.float64`: 64-bit precision
```

```
↳floating-point number type: sign bit, 11 bits exponent, 52 bits mantissa.
```

1.7 Conclusión

En esta práctica se calcularon diferentes medidas estadísticas descriptivas para analizar un conjunto de datos cuantitativos. Se estudiaron las medidas de dispersión, la asimetría y la curtosis, permitiendo comprender la variabilidad y la forma de una distribución.

Finalmente, se aplicaron estos conceptos al conjunto de datos **Iris**, identificando el comportamiento estadístico de algunas de sus variables y reforzando el uso de Python como herramienta para el análisis exploratorio de datos.

2 Ejercicio Extra: Dataset Iris

En este ejercicio se utilizará nuevamente el conjunto de datos **Iris** de Scikit-Learn.

El objetivo consiste en calcular la **media** y la **desviación estándar** de cada variable para cada una de las tres especies de flores (*setosa*, *versicolor* y *virginica*).

Posteriormente, estos resultados se representarán mediante un gráfico de barras de error (`plt.errorbar`), donde:

- La línea representa la **media**.
- Las barras verticales representan la **desviación estándar**.
- Cada color corresponde a una especie diferente.

```
[13]: # =====
# EJERCICIO EXTRA
# Media y desviación estándar por especie del dataset Iris
# =====

# Importar las bibliotecas necesarias.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# =====
# Cargar el conjunto de datos Iris.
# =====

iris = load_iris()

# Variables predictoras.
X = iris.data
```



```

# Clases del dataset.
y = iris.target

# Nombres de las especies.
especies = iris.target_names

# Nombres de las variables.
variables = iris.feature_names

# Posiciones del eje X.
x = np.arange(len(variables))

# =====
# Crear la figura.
# =====

plt.figure(figsize=(10,6))

# =====
# Calcular la media y desviación estándar para cada especie.
# =====

for i, especie in enumerate(especies):

    # Seleccionar únicamente los datos de la especie actual.
    datos = X[y == i]

    # Calcular la media de cada variable.
    media = datos.mean(axis=0)

    # Calcular la desviación estándar.
    desviacion = datos.std(axis=0)

    # Dibujar la línea con barras de error.
    plt.errorbar(
        x,
        media,
        yerr=desviacion,
        marker='o',
        linewidth=2,
        capsize=4,
        label=especie
    )

# =====
# Personalizar el gráfico.
# =====

```

```

plt.xticks(
    x,
    variables,
    rotation=20,
    fontsize=8
)

plt.xlabel("Variables")
plt.ylabel("Valor (cm)")

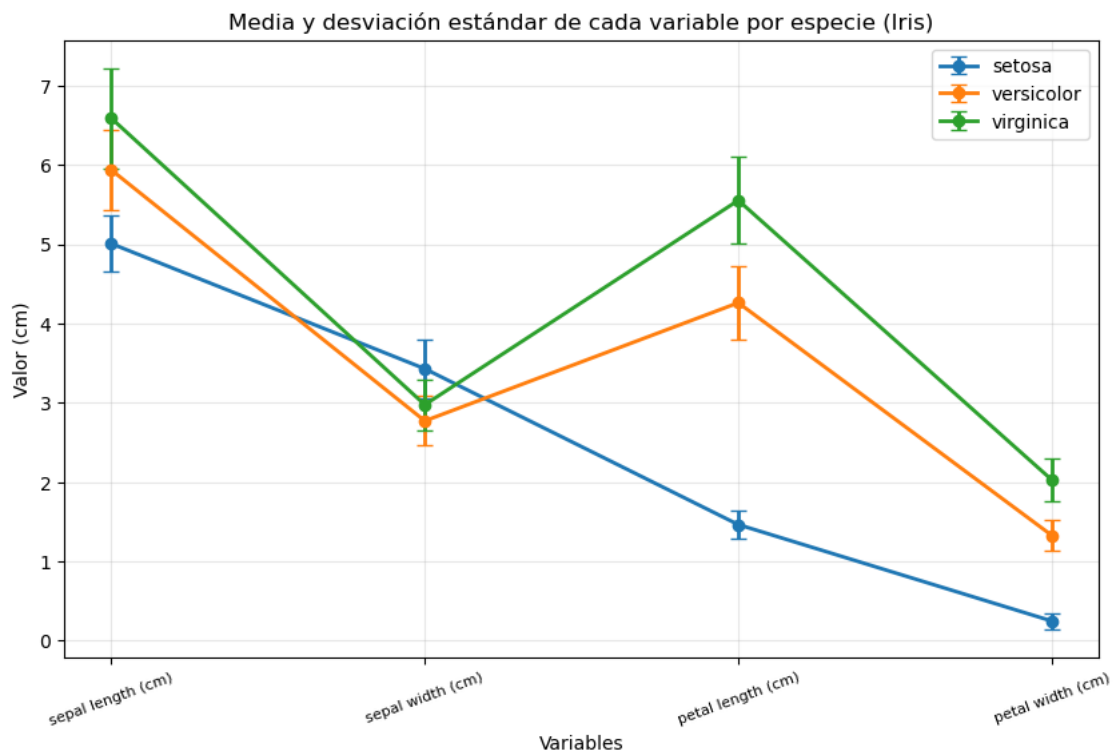
plt.title("Media y desviación estándar de cada variable por especie (Iris)")

plt.grid(alpha=0.3)

plt.legend()

# Mostrar el gráfico.
plt.show()

```



2.0.1 Interpretación

El gráfico muestra la media y la desviación estándar de cada variable para las tres especies de flores del conjunto de datos Iris.

Se observa que:

- **Setosa** presenta los pétalos más pequeños.
- **Virginica** posee, en promedio, los pétalos más largos y anchos.
- **Versicolor** presenta valores intermedios entre ambas especies.

Las barras de error permiten visualizar la variabilidad de cada característica dentro de cada especie.

[]:

